Ajaia Docs — AI-Native Development Workflow

Candidate: Sadat Shahriar Arko
Project: Ajaia Docs — Collaborative Document Studio
Live URL:https://ajaia-assessment-iota.vercel.app
Repository: https://github.com/Shahriar-Arko/AJAIA_Assessment.git

1. AI Tools Used

Google Gemini was used as an AI development assistant within VS Code throughout the project. It helped with application architecture, generating initial code structures, API development, database schemas, Tiptap integration, unit testing, and production deployment configuration.

2. Areas Where AI Accelerated Development

Full-Stack Setup and API Development

AI assistance helped generate the initial Next.js 16 App Router structure, typed CRUD endpoints, authentication helpers using HTTP-only cookies, and MongoDB connection caching in lib/mongodb.ts. This reduced the time spent on repetitive setup and allowed more attention to be given to access control and synchronization logic.

Document Processing and Templates

AI helped accelerate the integration of Mammoth.js for converting DOCX files into HTML. It also assisted in developing structured starter templates, including Product Specifications (PRD), Executive Meeting Notes, and Technical Research Briefs.

Test Development

AI was used to create initial Vitest test cases for role-based access control, password verification, and legacy collaborator data normalization.

3. AI-Generated Code That Was Modified or Replaced

AI-generated code was not accepted without testing. Several parts required manual debugging and changes to work correctly with the project's technical requirements.

PDF Export Implementation

The initial implementation used html2canvas with jsPDF. However, it encountered runtime errors when processing Tailwind CSS v4 color functions such as lab() and oklch(). I replaced this approach

with a DOM-traversal-based renderer using jsPDF, which extracts headings, lists, and paragraphs and places them into paginated A4 pages.

TypeScript Synchronization Logic

The initial background polling implementation in app/docs/[id]/page.tsx failed during next build with a TS2367 error. The issue was related to TypeScript control-flow narrowing across an asynchronous fetch operation. I introduced an isActivelyEditing() predicate to evaluate the editing state dynamically after asynchronous operations.

Next.js Route Parameters and Test Assertions

Dynamic API route handlers were updated to await context.params, following the Next.js 16 asynchronous route parameter requirements. I also corrected a Vitest password-hash assertion to match the actual 64-character hexadecimal SHA-256 implementation in lib/auth.ts, replacing an incorrect expectation of a salt:hash format.

4. Testing and Verification

The application was tested at multiple stages to check functionality, correctness, and production readiness.

Unit Testing: Executed npx vitest run to verify SHA-256 password hashing and verification, Owner/Editor/Viewer permissions, and collaborator schema normalization.

Production Build: Ran npm run build locally to check TypeScript compilation and Turbopack route generation before pushing changes to GitHub.

Live Deployment Testing: Tested the deployed application on Vercel, including user registration and login, DOCX/Markdown/TXT file importing, in-editor content appending, multi-tab document synchronization, collaborator permission changes, and A4 PDF downloads.

These checks helped verify the application's core functionality and identify issues that needed to be addressed before the final deployment.